

제10회 2026 미래에셋증권

AI Festival

예선 제출 · 기술 제안서

공시 Agent 과제

공시 QA Agent

제공 공시 코퍼스만을 근거로 자연어 질의에 답하는 4-Stage LangGraph Agent. 하이브리드 검색, 구조화 Fact 추출, 결정론적 계산, HyperCLOVA X 생성, 검증을 거쳐 근거 공시가 명시된 답변을 반환한다.

과제

공시 데이터 기반 검색·분석 Agent

저장소

dis-164

LLM

HyperCLOVA X

제출

2026.09

목차

01 문제 정의

02 제안 방법

03 시스템 구성

04 단계별 기능

05 Supervisor 분기

06 제안서 작성 시 표현 주의

07 사용자 시나리오

08 기대 효과

09 확장성

제안 한 줄

주최 측이 제공한 공시 원문·구조화 메타데이터를 전처리·인덱싱한 검색 코퍼스에서, 자연어 질의에 맞는 공시를 하이브리드 검색으로 찾고, 구조화 Fact 추출·결정론적 계산·HyperCLOVA X 답변 생성·4단계 검증을 거쳐, 근거 공시가 명시된 정확한 답변을 반환하는 **4-Stage LangGraph Agent**를 제안한다.

1. 문제 정의

1.1 배경 — 공시 정보 접근의 한계

국내 상장기업 공시는 2023년 1월 ~ 2026년 1분기 동안 수천 건의 정기·수시·지분 공시로 축적되어 있다. 매출·영업이익·설비투자·자금조달·계약·지분변동 등 투자·분석에 필수 정보가 담겨 있으나, 아래 한계가 있다.

문제	구체적 현상	영향
정보 분산	한 기업의 한 지표가 사업보고서·분기보고서·수시공시에 흩어짐	“2025년 연결기준 매출”처럼 단순 질의도 여러 문서를 열어야 함
형식 비정형	DART HTML/XML 표, 정정 공시, 후속 공시가 혼재	키워드 검색만으로는 정확한 수치·기간·기준을 찾기 어려움
비교·연산 필요	“A와 B 중 설비투자가 큰 기업”, “매출 증감률”, “유형별 자금조달 정리”	단일 문서 조회를 넘어 다중 문서 종합·계산이 필요
환각 위험	일반 LLM은 공시에 없는 수치·예측을 생성할 수 있음	금융 도메인의 신뢰성·규정 준수 요구와 충돌

1.2 과제 목표

제공 공시 코퍼스(2023.01 ~ 2026 Q1)만을 입력으로, 자연어 질의에 대해 검증 가능한 수치·사실과 근거 공시를 함께 반환하는 공시 QA Agent를 구축한다.

1.3 무엇을 (What)

배경의 4가지 한계를 해결하기 위해, 입력 x 를 출력 y 로 바꾸는 문제를 푼다.

$$x = (q, D)$$

변수	내용	구체적 데이터
q	자연어 질의	평가 API의 question (예: “OO기업 2025년 연결기준 매출액은?”)
D	제공 공시 코퍼스	공시 원문, universe.csv , manifest.jsonl , 사전 구축 인덱스 (chunk_index SQLite + Chroma 벡터)

배경 문제	이 과제에서 풀 것
정보 분산	q에 맞는 공시 자동 검색
형식 비정형	표·본문에서 수치·기간·기준 구조화 추출
비교·연산 필요	다중 문서 Fact 기반 비교·계산

배경 문제	이 과제에서 풀 것
환각 위험	근거 없는 답변 차단, 근거 공시 필수

제약

- 제공 코퍼스만 사용
- HyperCLOVA X만 LLM 사용
- 확인 불가 시 추측 금지

1.4 어떻게 (How)

전체 함수:

```
y = f(q, D; Theta)

Theta = 파이프라인 설정 (모델 weight 아님)
E5, HyperCLOVA X는 frozen (fine-tune 없음)
```

4단계 합성:

```
f = h4( h3( h2( h1(q), D ) ) )

intent      = h1(q)           # Interpreter
docs        = h2(intent, D)   # Retriever
facts, draft = h3(intent, docs) # Reasoner
answer      = h4(draft, facts) # Validator
```

함수	컴포넌트	역할	배경 대응
h1	Interpreter	질의 슬롯 추출, manifest_filter 생성	검색 범위 축소
h2	Retriever	SQL 메타 필터 + keyword/vector hybrid 검색	필요 문서 회수
h3	Reasoner	표 파싱, Fact 추출, 계산, 답변 초안 (질의 조건과 exact match Fact만 사용)	표 처리, 연산
h4	Validator	수치·출처·semantic 검증 (Grounding 위반 시 fail-closed)	환각 차단

최적화 목표 (L)

답이 얼마나 좋은지를 아래 4가지로 평가하고, 이를 합쳐 목표 함수 L이라고 한다.

```
L = 정확성 + 근거 완전성 + 요구사항 충족 - 환각
  = L_acc + L_evid + L_req + L_hall
```

파이프라인 설정(Theta)은 모델 weight를 학습하지 않고, gold Q&A로 L을 측정한 뒤 더 나은 설정을 고른다.

목표: $\text{Theta}^* = \operatorname{argmin} L(y, y^*)$

의미: L이 가장 작은 설정 Theta를 선택한다.

항목	무엇을 보는가	대응 배경	관련 단계
L_acc (정확성)	수치·순위·비교가 맞는지	형식 비정형	Reasoner, Validator
L_evid (근거 완전성)	필요한 공시가 검색됐는지	정보 분산	Interpreter, Retriever
L_req (요구사항 충족)	질문이 요구한 항목을 다 담았는지	비교·연산 필요	Interpreter, Reasoner
L_hall (환각)	공시에 없는 내용을 넣지 않았는지	환각 위험	Reasoner, Validator

운영 원칙 (fail-closed)

Validator(h4)에서 답이 Fact·출처와 맞지 않으면 답을 보내지 않고 “공시에서 확인되지 않음”으로 응답한다.

틀린 답을 내는 것보다 답하지 않는 것을 선택한다. L_hall과 L_acc를 지키기 위한 규칙이다.

파라미터 (Theta) 정의

Theta는 AI를 다시 학습하는 값이 아니라, 공시 검색·숫자 추출·답변 검증 방식을 조정하는 설정이다. E5와 HyperCLOVA X는 그대로 두고, 이 설정만 바꿔 평가 점수(L)를 올린다.

본 과제는 모델 fine-tune이 아니라, **frozen E5·HyperCLOVA X** 위에서 Theta를 평가 지표 기준으로 선택하는 문제이다.

Theta 항목	예시	고정 / 튜닝
인덱스	청킹, chunk_index, Chroma	사전 구축 후 고정
검색	hybrid 가중치, top-k, Reranker on/off	튜닝
파서·규칙	structured-parser, metric_registry	튜닝
Grounding	strict grounding gate	튜닝
LLM 설정	프롬프트, max_tokens	설정
E5, HyperCLOVA X	임베딩·생성 모델	frozen

튜닝 방법 (5단계)

1. gold Q&A로 현재 L 측정

- 미리 준비한 질문·정답 쌍(대회 참고 질의, 내부 테스트)으로 시스템을 돌린다.
- 정확성, 근거, 요구사항, 환각 네 가지(L) 중 어디가 약한지 본다.

2. Stage별 실패 원인 분류

실패 유형	주로 보는 단계	예시
공시를 못 찾음	h1, h2	잘못된 기간·문서 검색
숫자를 잘못 읽음	h3	표 헤더를 값으로 오인
답은 썼는데 검증 실패	h4	근거 없는 수치

3. Theta 한 부분만 변경 후 L 재측정 (ablation)

- ablation = 설정을 **하나만** 바꿔서 효과를 보는 것 (예: Reranker만 켜기, top-k만 100→200)
- 여러 개를 한꺼번에 바꾸면 무엇이 도움이 됐는지 알 수 없다.

4. 개선된 설정 채택

- L이 나아진 Theta 조합을 채택한다.
- 나아지지 않으면 원래 설정을 유지한다.

5. 회귀 테스트로 고정

- 회귀 테스트 = 예전에 맞았던 질문이 다시 틀리지 않는지 확인하는 테스트
- 새 설정이 한 문제만 고치고 다른 문제를 깨지 않았는지 검증한 뒤 고정한다.

구성요소	방식	영향 L
E5	multilingual-e5-large, frozen	L_evid
HyperCLOVA X	API, fine-tune 없음	L_req, L_hall
인덱스	칭킹 + SQLite + Chroma	L_evid
파서·규칙	metric_registry, parser	L_acc
검색	hybrid, candidate limit	L_evid

1.5 결과물 (Output)

```
y = (answer, retrieved_context, think_trace)
```

필드	내용
answer	검증 통과 답변 + 근거 공시
retrieved_context	사용한 검색 문서
think_trace	h1~h4 추론 과정

API: GET /answer → 5필드 JSON (question_id , question 포함)

성공 기준

- L_acc, L_evid, L_req, L_hall 충족
 - 모든 답변에 근거 공시 표시
 - 확인 불가 시 한계 명시 (무리한 확답 없음)
-

2. 제안 방법

2.1 전체 접근

공시 QA를 단일 LLM 문제가 아니라 **검색 → 추출 → 계산 → 검증** 4단계 문제로 해결한다.

3가지 원칙

- 1. **Stage 분리** — Interpreter / Retriever / Reasoner / Validator
- 2. **결정론 우선** — 수치는 Fact·계산, LLM은 문장 생성·검증만
- 3. **Fail-closed** — 근거·검증 실패 시 "공시에서 확인되지 않음"

2.2 LangGraph 선택 이유

질의마다 검색 범위, Fact 수, 계산 유형, 검증 결과가 달라지므로 **상태 기반 분기·재시도**가 필요하다.

구성	역할
AgentState	intent, retriever_result, facts, answer, think_trace
Stage 노드	각 hn 실행, partial update 반환
Supervisor	재검색, 답변 재생성, 종료만 허용

대안	한계	LangGraph
단일 LLM	근거·수치·검증 혼재	Stage 분리
if/else 파이프라인	재시도·분기 하드코딩	상태머신
다른 Agent 프레임워크	스택 맞춤 필요	h1~h4와 1:1 대응

2.3 데이터 준비

입력	작업	결과
DART 공시 원문	청킹, 메타 추출	SQLite chunk_index (약 115만 행)
청크 텍스트	E5 임베딩	Chroma 벡터 (약 80만 개)
기업·공시 정보	정규화	universe.csv, manifest.jsonl

인덱스는 **read-only 서빙**. `/answer` 요청 시 수정하지 않는다.

2.4 Stage별 처리

Interpreter (h1)

항목	내용
입력	question, universe.csv, manifest.jsonl
방법	슬롯 추출 → manifest_filter. 복합 질의는 query_plan, analysis_plan
출력	intent, route
개선 L	L_evid, L_req

Retriever (h2)

항목	내용
입력	manifest_filter, intent, SQLite, Chroma
방법	SQL 필터 → keyword + vector hybrid → merge → (선택) Reranker
출력	cited_documents
개선 L	L_evid

Reasoner (h3)

항목	내용
입력	cited_documents, intent, analysis_plan
방법	표 파싱 → Fact → 계산 → (필요 시) event linker → HyperCLOVA 답변
출력	facts, calculations, citations, draft
개선 L	L_acc, L_req, L_hall

Validator (h4)

항목	내용
입력	draft, facts, citations, intent
방법	수치 검증 → 출처 검증 → semantic 검증
출력	최종 answer 또는 fail-closed
개선 L	L_acc, L_hall

재생성 최대 1회. 재검증 실패 시 fail-closed.

2.5 질의 유형별 처리

유형	예시	강조	처리
검색·추출	OO기업 2025년 연결기준 매출액	h2, h3	Fact lookup
비교·연산	A와 B 중 설비투자 큰 기업	h1, h3	subquery, rank, 증감률
복합 추론	계약 체결 후 해지 여부	h2, h3	event linker

2.6 안전·신뢰성 확보

방법	내용	평가 항목
Grounding gate	exact match Fact만 사용	근거 기반, 정확성
Validator 3중 검증	수치, 출처, semantic	환각 방지
Route 분기	unsafe, unanswerable	안전성, 정보한계
Fail-closed	검증 실패 시 한계 명시	환각 방지
Citation	근거 공시 필수	근거 완전성
think_trace	Stage별 추론 기록	추론 논리성

3. 시스템 구성

구성 설명

본 시스템은 외부 평가자의 공시 질의를 FastAPI 공개 API로 수신하고, 단일 Python 프로세스 내부의 LangGraph 기반 4단계 Agent 파이프라인에서 처리한다. Interpreter가 질문의 기업·기간·공시 유형·지표·비교 조건을 구조화한 뒤, Retriever가 제공 공시 코퍼스의 메타데이터 필터와 키워드·벡터 검색을 결합해 근거 문서를 찾는다. Reasoner는 검색 결과에서 Fact를 추출하고 필요한 계산·비교·정정·후속 공시 연결을 수행하며, Validator는 답변의 수치와 citation, 의미적 근거성을 검증한다. 검색 인덱스와 E5 임베딩 모델은 읽기 전용 로컬 자원으로 사용하고, CLOVA Studio는 답변 생성·의미 검증·선택적 reranking이 필요한 경우에만 연동한다.

3.1 구성 요소

구성 요소	역할	기술·구현 기준
Public API	평가자의 질의를 수신하고 최종 JSON을 반환	FastAPI, <code>/health</code> , <code>/ready</code> , <code>/answer</code>
StagePipeline	요청별 초기 상태를 만들고 그래프 실행을 시작	<code>integration.service:StagePipeline</code>
Root LangGraph	단계 실행 순서와 조건부 분기를 관리	<code>integration.graph:build_graph</code>
Interpreter	질문을 정규화하고 기업·기간·문서 유형·지표·계산 조건을 추출	<code>interpreter</code> 모듈, corpus metadata 사용
Deterministic Supervisor	현재 phase와 결과를 바탕으로 다음 실행 노드를 선택	기본 동작은 규칙 기반 정책
Retriever	후보 필터링, 키워드·벡터 검색, 병합·reranking 수행	SQLite + E5/HNSW + 선택적 CLOVA reranker
Reasoner	원문·표·XML/HTML에서 Fact를 추출하고 계산·비교·이벤트를 수행	결정론적 분석 + 선택적 CLOVA 답변 생성
Validator	답변의 수치, citation, semantic grounding을 검증	실패 시 제한 응답 또는 1회 재생성
제공 코퍼스	기업·문서·기간·공시 유형의 기준 메타데이터 제공	<code>universe.csv</code> , <code>manifest.jsonl</code> , 규칙 JSON
SQLite Chunk Index	메타데이터 조건에 맞는 문서 후보와 텍스트 chunk 조회	serving 시 read-only
Chroma/HNSW Index	E5 임베딩 기반 의미 검색 수행	serving 시 persisted index read-only
E5 Model Cache	검색 질의를 1024차원 벡터로 변환	<code>intfloat/multilingual-e5-large</code> , <code>fastembed/ONNX</code>
CLOVA Studio	답변 생성, semantic 검증, 선택적 reranking	환경 플래그로 기능별 활성화

3.2 설계에서 강조할 점

1. 단일 배포 프로세스와 논리적 단계의 분리

Interpreter·Retriever·Reasoner·Validator는 독립 서버가 아니라, 하나의 FastAPI 프로세스 안에서 실행되는 논리적 구성 요소다.

2. 검색 근거의 이중화

SQLite 메타데이터 필터로 검색 범위를 먼저 제한한 뒤, 키워드 검색과 E5/HNSW 벡터 검색을 결합한다.

3. 읽기 전용 인덱스 운영

평가 요청 중에는 공시 원문과 SQLite·Chroma/HNSW 산출물을 수정하거나 재색인하지 않는다.

4. 선택적 외부 모델 연동

CLOVA Studio는 모든 제어를 담당하는 Supervisor가 아니라, 답변 생성·semantic 검증·reranking 등 지정된 지점에서 선택적으로 사용된다.

5. 근거 기반 안전성

검색 문서와 Fact가 부족하면 추측으로 답변하지 않고 clarification, unanswerable 또는 fail-closed 경로로 전환한다.

사용자 질의는 API 수신 후 `AgentState` 로 관리되며, 모든 주요 단계는 Supervisor를 거쳐 다음 단계로 이동한다. Interpreter는 질의의 의미와 검색 조건을 구조화하고, Retriever는 조건에 맞는 후보를 좁힌 후 키워드·벡터 검색 결과를 결합한다. 근거 문서가 확보되면 Reasoner가 수치와 사실을 추출해 계산·비교·이벤트 연결을 수행하고, Validator가 최종 답변의 수치와 인용 근거를 검증한다. 검색 결과가 부족하거나 답변 검증이 실패하는 경우에는 제한된 횟수의 재검색·재생성을 수행하며, 끝까지 근거가 확보되지 않으면 안전한 실패 응답으로 종료한다.

4. 단계별 기능

4.1 요청 수신 및 실행 상태 초기화

/answer 는 question_id 와 question 을 받고, 최초 요청 시 pipeline을 지연 초기화한다. StagePipeline 은 질문 원문을 original_question 에 보존하고, 각 단계의 결과·경고·검색 이력·재시도 횟수를 AgentState 에 누적한다. 최종 응답은 평가 API 규격에 맞춰 5개 문자열 필드로 변환한다.

4.2 Interpreter의 질의 구조화

질의를 정규화한 뒤 기업명·종목코드·섹터·기간·분기·공시 유형·지표·연결/별도 기준·정정 여부·비교 및 계산 조건을 추출한다. 이 정보는 manifest_filter , search_query , query_plan 으로 변환되어 Retriever의 검색 범위를 제한한다.

위험 질의, 범위 밖 질의, 필수 정보가 부족한 질의는 검색 전에 각각 fail-closed / unanswerable / clarification 경로로 보낸다.

4.3 복합 질의 분석 계획

여러 기업·연도·문서·계산 조건이 포함된 질문은 Calculation Planner가 필요한 하위 요구사항과 계산 순서를 만든다. 각 요구사항은 별도의 검색·분석 단위로 처리한 뒤 Reasoner 단계에서 결과를 합친다.

deterministic planner를 우선 사용하고, 설정된 경우에만 CLOVA JSON 호출로 계획 보완을 수행한다.

4.4 Hybrid Retrieval

1. SQLite Chunk Index에서 기업·기간·공시 유형 등 메타데이터 조건에 맞는 후보를 좁힌다.
2. 키워드 검색과 E5 임베딩 기반 HNSW 벡터 검색을 병렬 수행한다.
3. 문서 ID 기준으로 결과를 병합한다.
4. 선택적 reranker를 적용한 뒤, 연도·문서 다양성 및 relevance를 반영해 최종 근거 문서를 확정한다.

HNSW는 요청 시 재생성하지 않고, 사전에 준비된 persisted index를 읽기 전용으로 조회한다.

4.5 Reasoner의 근거 기반 분석

검색 문서의 본문·표·XML/HTML에서 Fact를 추출하고, 수치·단위·기간·통화·연결/별도 기준을 정규화한다. 질문 유형에 따라 증감률·비중·합계·평균·순위 등을 결정론적으로 계산하고, 비교 대상이나 계약 체결·해지, 원공시·정정공시·후속공시의 관계를 연결한다. 이후 답변에 사용할 citation과 context를 구성한다.

4.6 Validator의 품질·안전성 검증

Validator는 답변의 숫자가 Fact 또는 계산 결과와 일치하는지, citation이 실제 검색 문서에 존재하는지, 설명이 근거 범위를 벗어나지 않는지를 검사한다.

검증 실패 시 답변 재생성 기능이 활성화되어 있으면 **최대 1회** 재생성한 뒤 다시 검증한다. 재생성 후에도 실패하면 근거 없는 답변을 반환하지 않고 제한된 실패 응답으로 종료한다.

5. Supervisor 분기

발생 시점	조건	다음 흐름
Interpreter 이후	일반 질의	Retriever 실행
Interpreter 이후	계산·비교·다중 조회 필요	Calculation Planner → Retriever
Interpreter 이후	unsafe, 범위 밖, 필수 정보 부족	Fail-closed / Clarify / Unanswerable → Validator
Retriever 이후	인용 문서 존재	Reasoner 실행
Retriever 이후	문서 없음, 재시도 가능	Search Retry → Retriever
Retriever 이후	문서 없음, 재시도 소진	Unanswerable → Validator
Reasoner 이후	분석 결과 생성	Validator 실행
Validator 이후	검증 통과	최종 응답
Validator 이후	검증 실패, 재생성 가능	Answer Regeneration → Validator
Validator 이후	검증 실패, 재생성 소진	Fail-closed 최종 응답

최종 API 응답

```
{
  "question_id": "질의 ID",
  "question": "질의 원문",
  "retrieved_context": "실제 답변에 사용한 공시 근거",
  "think_trace": "검색·계산·검증 과정의 요약",
  "answer": "최종 답변 또는 제한된 실패 안내"
}
```

6. 제안서 작성 시 표현 주의

- `Interpreter`, `Retriever`, `Reasoner`, `Validator` 를 각각 독립 마이크로서비스라고 표현하지 않는다. 현재는 하나의 FastAPI 프로세스 내부 모듈이다.
 - Supervisor는 현재 기본 실행 경로에서 LLM 기반 Agent가 아니라 **DeterministicSupervisor** 다. LLM이 모든 단계를 ToolNode로 직접 제어한다고 쓰지 않는다.
 - HNSW는 serving 요청 중 생성·재색인하는 저장소가 아니라, 사전에 준비된 **read-only 인덱스**다.
 - `retriever/ingestion/dart/` 는 오프라인 데이터·인덱스 준비 코드이며, 평가 질의마다 OpenDART나 외부 금융 API를 호출하는 구조로 설명하지 않는다.
 - CLOVA Studio는 기능별 환경 설정에 따라 답변 생성·semantic 검증·reranking 등에 **선택적으로** 사용된다. 검색 결과와 결정론적 계산·검증 경로를 CLOVA 호출 하나로 뭉뚱그리지 않는다.
 - `reasoner/orchestration/langgraph_graph.py` 의 one-node compatibility graph는 canonical root graph의 하위 그래프가 아니다. 시스템 구성도에는 별도 구성요소로 넣지 않는다.
-

7. 사용자 시나리오

7.1 구성 기준

대회 자료는 평가 질의가 난이도 상·중·하와 Closed/Open-ended 유형을 혼합한다고 안내한다. 따라서 사용자 시나리오도 네 가지 유형을 대표하도록 구성한다.

각 시나리오는 서로 다른 핵심 기능을 보여주며, **실제 평가문제를 재현하는 것이 아니라** 시스템 처리 범위를 설명하기 위한 예시다.

유형	대표 질의	핵심적으로 보여주는 기능
쉽고 명확한 Closed	특정 기업의 2025년 연결기준 매출액은 얼마인가?	단일 수치 검색·Fact 추출·숫자 검증
쉽고 명확한 Open	특정 기업의 2026년 1분기 주요 투자 계획을 정리해줘.	문서 내용 요약·항목별 citation·semantic 검증
어렵고 복합적인 Closed	특정 기업이 2025년에 체결한 주요 계약 중 이후 해지된 계약이 있는가?	체결·해지 공시 연결·존재 여부 판단·이력 citation
어렵고 복합적인 Open	특정 기업의 2023년과 2025년 사업보고서를 비교해 핵심 사업 변화를 설명해줘.	다년 문서 비교·Fact 정렬·변화 설명·근거 완결성

호출 예시:

```
curl -sG "http://HOST:8000/answer" \
  --data-urlencode "question_id=Q-001" \
  --data-urlencode "question=여기에 질문 붙여넣기" \
  | python3 -m json.tool
```

7.2 쉽고 명확한 Closed — 단일 수치 조회

사용자 질문

현대자동차의 2025년 연결 기준 매출액은 얼마인가?

사용자 목적

특정 기업·연도·연결 기준·단일 지표를 빠르게 확인하고, 해당 수치가 어느 공시에서 추출되었는지 확인한다.

시스템 처리

- 사용자가 GET /answer 로 question_id 와 질문을 전송한다.
- Interpreter가 기업, 2025년, 매출액, 연결 기준을 추출하고 manifest_filter 를 만든다.
- Deterministic Supervisor가 일반 검색 경로를 선택한다.

4. Retriever가 정공시 후보를 메타데이터로 제한한 뒤 키워드·E5/HNSW 검색으로 관련 Fact를 찾는다.
5. Reasoner가 매출액, 단위, 기준 기간, 연결 여부를 정규화하고 citation을 만든다.
6. Validator가 답변의 숫자와 근거 문서가 일치하는지 검증한다.

사용자가 받는 결과

매출액, 기준 기간, 단위, 연결 기준, 근거 공시가 포함된 짧고 명확한 답변을 받는다.

이 시나리오는 `Interpreter → 단일 Retriever → Fact 추출 → numeric/citation validation` 능력을 보여준다.

7.3 쉽고 명확한 Open — 공시 내용 요약

사용자 질문

현대자동차의 2026년 1분기 주요 투자 계획을 공시 기준으로 정리해줘.

사용자 목적

하나의 수치가 아니라 분기보고서 또는 관련 공시에서 여러 투자 계획과 세부 내용을 추출해 이해하기 쉬운 항목으로 정리한다.

시스템 처리

1. Interpreter가 기업, 2026년 1분기, 투자 계획, 요약·설명 요청을 추출한다.
2. Retriever가 분기보고서와 관련 공시를 기간·기업·공시 유형으로 필터링하고, 키워드·벡터 검색으로 관련 문서를 수집한다.
3. Reasoner가 본문·표·구조화 데이터에서 투자 대상, 목적, 금액·기간 등 확인 가능한 Fact를 추출한다.
4. 질문이 요구한 항목별로 Fact를 묶고, 각 항목에 근거 citation을 연결해 답변 초안을 만든다.
5. CLOVA 답변 생성이 활성화된 경우 근거 기반 자연어 요약을 생성하며, 그렇지 않거나 호출에 실패하면 결정론적 grounded 답변으로 fallback한다.
6. Validator가 요약에 포함된 수치·사실과 citation, 근거 범위 밖의 표현을 검증한다.

사용자가 받는 결과

투자 대상·목적·금액·기간 등 공시에서 확인되는 항목을 구조화한 요약과 각 항목의 근거 공시를 받는다.

이 시나리오는 `text/fact extraction → 항목별 답변 구성 → semantic/citation validation` 능력을 보여준다.

7.4 어렵고 복합적인 Closed — 계약 체결·해지 이력 연결

사용자 질문

특정 기업이 2025년에 체결한 주요 계약 중 이후 해지된 계약이 있는가? 있다면 계약명과 해지일을 알려줘.

사용자 목적

체결 공시와 이후 해지 공시가 같은 계약을 가리키는지 확인하고, 최종적으로 해지 계약의 존재 여부와 목록을 정확하게 확인한다.

시스템 처리

1. Interpreter가 체결 기간, 후속 기간, 계약 공시, 해지 이벤트, 계약명·해지일이라는 출력 항목을 추출한다.
2. 복합 문서 질의로 판단해 `manifest_filter`, `query_plan`, `include_chain` 조건을 만든다. 여러 검색 요구사항으로 분해할 필요가 있으면 Calculation Planner를 거치고, 단일 계약 이력 조회가 가능하면 Retriever로 바로 이동한다.
3. Retriever가 기업·접수일·공시 유형을 기준으로 후보를 좁히고, 키워드·E5/HNSW 검색 결과를 병합한다.
4. Reasoner가 계약명, 계약 당사자, 원공시 접수번호 또는 정확한 계약 식별 정보를 사용해 체결 공시와 해지 공시를 연결한다.
5. 식별 근거가 충분한 경우에만 해지 이벤트로 확정하고, 계약명·체결일·해지일과 연결된 citation을 구성한다.
6. Validator가 존재 여부, 계약명, 날짜, 원공시·후속공시 관계를 검증한다.

사용자가 받는 결과

해지된 계약의 존재 여부와 계약별 계약명·체결일·해지일, 체결·해지 공시의 연결 근거를 받는다. 연결 근거가 부족하면 계약을 임의로 같은 건으로 묶지 않고 확인 불가로 처리한다.

이 시나리오는 `multi-document retrieval → event linking → closed conclusion → provenance validation` 능력을 보여준다.

7.5 어렵고 복합적인 Open — 다년 사업 변화 비교

사용자 질문

특정 기업의 2023년과 2025년 사업보고서를 비교해 핵심 사업 변화를 설명해줘.

사용자 목적

서로 다른 연도의 사업보고서에서 사업 부문·제품·지역·투자 방향 등 비교 가능한 Fact를 추출하고, 단순 나열이 아니라 변화의 내용과 근거를 설명한다.

시스템 처리

1. Interpreter가 기업, 2023년·2025년, 사업보고서, 핵심 사업 변화라는 비교 조건을 추출한다.
2. Calculation Planner가 두 연도의 사업보고서와 비교 대상 항목을 각각 검색·분석 요구사항으로 분해한다.
3. Retriever가 연도와 보고서 유형을 분리해 검색하고, 각 연도의 문서를 독립적으로 수집한다.
4. Reasoner가 각 연도의 사업 Fact를 추출·정규화한 뒤 사업 부문, 제품, 지역, 투자 등 비교 축별로 정렬한다.
5. 두 시점의 공통점·변경점·신규 또는 축소된 내용을 근거 문서와 함께 설명하는 답변을 만든다.
6. Validator가 질문이 요구한 두 연도를 모두 반영했는지, 설명이 각 연도 문서의 근거를 벗어나지 않는지, citation이 충분한지 검증한다.

사용자가 받는 결과

2023년과 2025년의 사업 구조를 항목별로 비교한 설명, 핵심 변화 요약, 변화별 근거 공시를 받는다. 근거가 없는 원인이나 미래 전망은 추가하지 않는다.

이 시나리오는 multi-year retrieval → comparison analysis → open-ended explanation → completeness/semantic validation 능력을 보여준다.

7.6 공통 예외 시나리오

근거 문서가 충분하지 않은 경우

Retriever가 인용 가능한 문서를 찾지 못하면 Supervisor가 `retry_search` 를 선택하고 검색어를 보정해 한 번 재검색한다. 재검색 후에도 근거가 없으면 `unanswerable` 경로로 전환하고, Validator가 제한된 실패 응답을 만든다. 시스템은 검색되지 않은 수치나 공시 밖의 내용을 추측하지 않는다.

범위 밖 또는 위험한 질문인 경우

Interpreter가 제공 코퍼스 범위 밖의 기업·기간, 투자 추천·주가 전망 등 처리할 수 없는 질문을 식별하면 정상 검색을 시작하지 않는다. 질문 상태에 따라 clarification, unanswerable 또는 fail-closed 경로로 보내고, 근거가 없다는 사실과 확인 가능한 범위를 안내한다.

답변 검증에 실패한 경우

Validator가 숫자·citation·semantic 검증 중 하나라도 통과시키지 못하면, 재생성 기능이 활성화된 경우 답변을 최대 1회 다시 생성한 뒤 재검증한다. 재생성 후에도 실패하면 검증되지 않은 답변을 반환하지 않고 안전한 제한 응답으로 종료한다.

7.7 시나리오 근거

이 시나리오는 다음 기존 문서의 내용을 조합한 것이며, 별도 기능을 가정해 추가하지 않았다.

- docs/analysis/step1-code-analysis.md 2.1절: `/answer` 진입점과 lazy pipeline 초기화
- docs/analysis/step1-code-analysis.md 3절 및 5.1~5.9절: pipeline 조립, Interpreter·Retriever·Reasoner·Validator의 실제 처리
- docs/analysis/step2-code-analysis.md 4~6절: Supervisor route, planner·검색 retry·재생성·종료 조건
- docs/analysis/step5-code-analysis.md 2절 및 5절: 단일 사용자 질의 흐름과 단계별 데이터 이동
- docs/architecture/c4-context.md : 평가자, Agent, 로컬 공시 자원, CLOVA Studio의 관계
- docs/architecture/c4-components-pipeline.md : 각 component의 책임과 실제 코드 연결

8. 기대 효과

8.1 사용자·평가 관점

효과	데이터·작업	결과
정확한 수치 답변	공시 청크 → Fact 추출 → 결정론적 계산	매출·비중·증감률·순위 질의에 검증 가능한 수치 제공
근거 추적 가능	검색 청크 + document_id citation	모든 답변에 “어느 공시·어느 기간” 근거가 함께 표시
환각 억제	Stage4 수치·출처·의미 검증, fail-closed	공시에 없는 내용은 생성하지 않고 한계를 명시
수동 탐색 시간 절감	manifest 필터 + 하이브리드 검색	다수 공시를 직접 열람하지 않고 질의 한 번으로 핵심 정보 도출
복합 질의 처리	query_plan · analysis_plan 다중 subquery	기업 비교, 유형별 정리, 정정·해지 이력 등 다단계 분석 자동화

8.2 대회 평가 지표 대응

- **정확성·요구사항 충족** — Intent 슬롯(기업·기간·지표·기준)과 Fact gate로 질의 조건과 답변을 1:1 대응
- **근거 완전성** — retrieved_context 에 답변 필수 문서를 포함하고, Stage4에서 citation 누락 시 차단
- **추론 논리성** — 계산식·비교 결과를 Fact 체인(input_fact_ids)으로 보존해 추론 경로를 재현 가능
- **안전성·정보한계 대응** — unsafe · unanswerable route와 제한적 실패 응답으로 무리한 확답 방지

8.3 실무 활용

- **리서치·IR** — 특정 기업·기간의 재무·투자·계약 정보를 빠르게 조회
- **비교 분석** — 동일 지표·기간 기준으로 기업 간 순위·규모 비교
- **컴플라이언스·검증** — 답변 수치가 실제 공시와 일치하는지 근거 문서로 즉시 대조 가능

9. 확장성

9.1 기능 확장

확장 항목	현재 구조	확장 방법
지표·계산	<code>metric_registry</code> + calculation whitelist	새 <code>metric·operation</code> 등록만으로 증감률·비중·합계 등 질의 유형 추가
공시 유형	<code>manifest_filter</code> 의 <code>doc_group</code> · <code>doc_subtype</code>	신규 공시 유형 메타만 반영하면 Stage1 필터 · Stage2 검색에 자동 연동
질의 복잡도	<code>query_plan</code> , <code>analysis_plan</code> DAG	subquery·계산 step 추가로 N개 기업·N개 기간 비교 확장
이력 분석	event linker (정정·후속 공시)	접수번호·계약명 매칭 규칙 강화로 변경 이력·해지 추적 범위 확대
검색 백엔드	SQLite + Chroma hybrid (production)	retrieval experiment sidecar로 FTS5·FAISS 등 대체 인덱스 A/B 테스트 후 교체

9.2 데이터·코퍼스 확장

- 기간·기업 추가 — 제공 코퍼스가 갱신되면 청킹·임베딩·인덱스 재빌드(`build_chunk_index`)만 수행. Stage1~4 코드 변경 최소화
- 인덱스 규모 — SQLite 메타 필터로 후보를 먼저 축소한 뒤 벡터 검색. 기업·기간 증가 시에도 검색 범위를 질의 조건으로 제한
- 부분 인덱스 — 벡터·문서 집합 불일치 시 `STAGE2_ALLOW_PARTIAL_INDEX` 로 점진적 전환 가능

9.3 시스템·운영 확장

영역	현재	확장 방향
배포	NCP 단일 EC2 + Block Storage	앱·인덱스 볼륨 분리로 스토리지·컴퓨트 독립 확장
성능	process-local cache (TTL/LRU)	동시 요청 증가 시 worker 수 조정. 인덱스는 read-only 공유
LLM	HyperCLOVA X 단일 provider	대회 규정 내 모델 버전·Reranker on/off만 조정 (fine-tune 없음)
검증	Stage4 numeric + citation + semantic	검증 규칙·metric gate 추가로 도메인별 정밀도 향상
관측	<code>think_trace</code> , stage별 trace	질의 유형·실패 원인별 모니터링으로 약점 stage 선별 개선

9.4 확장 시 유지 원칙

1. **Stage 책임 분리** — 검색(Stage2)·추출·계산(Stage3)·검증(Stage4)을 독립 확장
2. **결정론 우선** — 수치·비교는 코드로, LLM은 표현·요약에만 사용
3. **Fail-closed 유지** — 기능 추가 시에도 근거 없는 답변 생성 경로를 열지 않음
4. **평가 API 호환** — `GET /answer` 5필드 스키마 고정으로 외부 평가 시스템과 독립 운영